

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	N. Tharun Sai	Reg. No.:	192472382
Department:	CSE AI	Date:	01/08/20206

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyse how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

1. Signed 8-bit Addition and Subtraction using 2's Complement

Given:

- $A = +45$
- $B = -23$

Step 1: Convert to Binary

+45

$45 = 00101101$

-23

First write +23:

$23 = 00010111$

Take 2's complement:

- 1's complement = 11101000
- Add 1 = 11101001

Therefore,

$-23 = 11101001$

A) Addition (+45 + (-23))

00101101 (+45)

+ 11101001 (-23)

00010110

Ignore carry out.

Result = **00010110**

Decimal = **22**

Overflow Check

Overflow occurs when:

- Positive + Positive = Negative
- Negative + Negative = Positive

Here,

- One positive
- One negative

Therefore,

No Overflow

B) Subtraction (+45 - (-23))

Subtracting a negative means adding the positive.

$$45 - (-23)$$

$$= 45 + 23$$

$$+23 = \mathbf{00010111}$$

$$00101101$$

$$+ 00010111$$

$$01000100$$

$$\text{Result} = \mathbf{01000100}$$

$$\text{Decimal} = \mathbf{68}$$

Overflow

68 is within the 8-bit signed range:

$$-128 \text{ to } +127$$

Therefore,

No Overflow

Overflow Detection

Processor checks the carry into and carry out of the MSB.

Overflow occurs when

Carry into MSB $\neq$ Carry out of MSB

or

- Positive + Positive $\rightarrow$ Negative
- Negative + Negative $\rightarrow$ Positive

Otherwise,

No Overflow

Final Results

Operation	Binary Result	Decimal	Overflow
45 + (-23)	00010110	22	No
45 - (-23)	01000100	68	No

2. 4-bit Carry Look Ahead Adder (CLA)

Suppose

$$A = 1010$$

$$B = 0110$$

$$C_{in} = 0$$

Step 1: Generate (G) and Propagate (P)

Formula

$$G_i = A_i \times B_i$$

$$P_i = A_i \oplus B_i$$

Step 2: Carry Equations

$$C_1 = G_0 + P_0 C_{in}$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_{in}$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + \dots$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + \dots$$

Carries are calculated simultaneously.

Ripple Carry Adder Delay

Each carry waits for the previous carry.

Example:

$$C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$$

Total delay

$$4 \times \text{Gate Delay}$$

CLA Delay

Carries are generated in parallel.

Delay

≈ 2 Gate Delays

Comparison

Ripple Carry	Carry Look Ahead
Slow	Fast
Carry generated one by one	Carry generated together
Large delay	Very small delay
Simple circuit	More hardware

3. Booth's Algorithm (-6 × +5)

Given

Multiplicand (M) = -6

Multiplier (Q) = +5

4-bit representation

-6 =1010

+5 =0101

Initial values

A =0000

Q =0101

Q-1 =0

Booth Rules

Q0 Q-1	Operation
00	No operation
11	No operation
01	A = A + M
10	A = A - M

After every step,

Perform Arithmetic Right Shift.

Cycle 1

$Q_0Q_{-1}=10$

$A=A-M$

Shift

Cycle 2

$Q_0Q_{-1}=11$

No operation

Shift

Cycle 3

$Q_0Q_{-1}=01$

$A=A+M$

Shift

Cycle 4

$Q_0Q_{-1}=10$

$A=A-M$

Shift

Final Product

11100010

Decimal

-30

Since

$$-6 \times 5 = -30$$

Correct.

Why Booth's Algorithm is Efficient

- Handles signed numbers automatically
- Reduces repeated additions
- Groups consecutive 1s
- Faster than ordinary multiplication
- Widely used in processors

Components

Register	Purpose
A	Accumulator
Q	Multiplier
M	Multiplicand
Q-1	Booth bit

4. Division ($13 \div 3$)

Binary Numbers

Dividend = 13 = 1101

Divisor = 3 = 0011

(A) Restoring Division

Initial

A = 0000

Q = 1101

M = 0011

Step 1

Shift AQ left

Subtract divisor

If A negative

Restore A

Quotient bit = 0

Step 2

Shift

Subtract

Positive

Quotient bit = 1

Step 3

Shift

Subtract

Negative

Restore

Quotient bit =0

Step 4

Shift

Subtract

Positive

Quotient bit =1

Final

Quotient =0100

Remainder =0001

Decimal

$13 \div 3$

Quotient=4

Remainder=1

(B) Non-Restoring Division

Initial

A=0000

Q=1101

Algorithm

- If remainder positive → subtract divisor
- If remainder negative → add divisor
- No restoring after every negative result

Final

Quotient=0100

Remainder=0001

Comparison

Restoring	Non-Restoring
Restores after negative result	No restoring
More operations	Fewer operations
Slower	Faster
Simple	Efficient

Why non-restoring is Preferred

- Less hardware activity
- Fewer arithmetic operations
- Faster execution
- Used in modern processors

5. IEEE 754 Floating-Point Representation

Given

-13.25

Step 1: Convert to Binary

Integer

13 = 1101

Fraction

0.25 = 0.01

Binary

1101.01

Normalize

1.10101×2^3

Single Precision (32-bit)

Sign Bit

Negative

S=1

Exponent

Bias

127

Exponent

$3 + 127 = 130$

130=10000010

Mantissa

Remove leading 1

101010000000000000000000

Final Representation

1

10000010

101010000000000000000000

Double Precision (64-bit)

Sign

1

Exponent

Bias

1023

Exponent

1026

10000000010

Mantissa

1010100

Floating Point Addition

Suppose

13.25

+

2.5

Steps

1. Compare exponents.
2. Shift smaller mantissa.
3. Add mantissas.
4. Normalize result.
5. Round if needed.
6. Store sign, exponent and mantissa.

Precision Issues

- Some decimal numbers cannot be represented exactly in binary.
- Rounding occurs when mantissa exceeds its size.
- Small precision loss may occur during arithmetic operations.

Difference Between Single and Double Precision

Feature	Single Precision	Double Precision
Total bits	32	64
Sign bits	1	1
Exponent bits	8	11
Mantissa bits	23	52
Precision	About 7 decimal digits	About 15–16 decimal digits
Memory usage	Less	More

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Appropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation